# Software Requirements Specification

## Student–Course Many-to-Many Mapping System with Database-Based Role Security

---

## 1. Project Title

**Student Course Management System using Many-to-Many Mapping in Spring Boot**

---

## 2. Project Objective

The objective of this project is to build a clean, structured Spring Boot REST API that demonstrates:

1. Many-to-many mapping between `Student` and `Course`.
2. Database-based role security using `User` and `Role`.
3. DTO-based request and response handling.
4. Role-based API authorization.
5. Pagination with sorting.
6. Swagger/OpenAPI documentation.
7. Global exception handling.
8. Logging for important application operations.

---

## 3. Core Business Use Case

The system must manage students and courses.

Business relationship:

| Entity | Relationship |
|---|---|
| Student | A student can enroll in many courses |
| Course | A course can have many students |

Example:

| Student | Courses |
|---|---|
| Rahul Sharma | Java Full Stack, Spring Boot |

| Student | Courses |
|---|---|
| Priya Mehta | Spring Boot, Angular |

The project must implement a direct many-to-many relationship between `Student` and `Course`.

---

## 4. Security Use Case

The system must implement database-based role security.

Security relationship:

| Entity | Relationship |
|---|---|
| User | A user can have many roles |
| Role | A role can belong to many users |

The system must not use JWT.

The system must use Basic Authentication with users and roles stored in the database.

---

## 5. Technology Stack

The project must use:

| Layer / Purpose | Technology |
|---|---|
| Programming Language | Java |
| Framework | Spring Boot |
| REST API | Spring Web |
| ORM | Spring Data JPA / Hibernate |
| Database | MySQL |
| Security | Spring Security |
| Authentication Type | HTTP Basic Authentication |
| Role Storage | Database |
| Validation | Jakarta Validation |
| DTO Mapping | ModelMapper |

| Layer / Purpose | Technology |
|---|---|
| Documentation | Swagger / OpenAPI |
| Boilerplate Reduction | Lombok |
| Logging | SLF4J Logger |
| Build Tool | Maven |

---

## 6. Scope of the Project

The project must include:

1. Course management.
2. Student management.
3. Student-course assignment.
4. Student-course removal.
5. Fetching courses assigned to a student.
6. Fetching students enrolled in a course.
7. Pagination and sorting for students.
8. Pagination and sorting for courses.
9. Database-based users and roles.
10. Role-based access control.
11. Swagger documentation.
12. Global exception handling.
13. Logging in important layers.

---

## 7. Out of Scope

The following must not be implemented in this version:

1. JWT authentication.
2. Email verification.
3. Password reset.
4. Frontend UI.
5. Payment module.
6. Enrollment entity with grade/status.
7. File upload.
8. Advanced search filters.
9. Refresh tokens.
10. OAuth login.

---

# 8. Project Structure Requirement

The project must follow this exact structure:

```
src/main/java/com/swabhav/demo
│
├── DemoApplication.java
│
├── config
│   ├── AppConfig.java
│   ├── DataInitializer.java
│   ├── OpenApiConfig.java
│   └── SecurityConfig.java
│
├── controller
│   ├── CourseController.java
│   └── StudentController.java
│
├── dto
│   ├── CourseRequestDto.java
│   ├── CourseResponseDto.java
│   ├── PageResponseDto.java
│   ├── StudentRequestDto.java
│   └── StudentResponseDto.java
│
├── exception
│   ├── BadRequestException.java
│   ├── DuplicateResourceException.java
│   ├── GlobalExceptionHandler.java
│   └── ResourceNotFoundException.java
│
├── model
│   ├── AppUser.java
│   ├── Course.java
│   ├── Role.java
│   └── Student.java
│
├── repository
│   ├── CourseRepository.java
│   ├── RoleRepository.java
│   ├── StudentRepository.java
│   └── UserRepository.java
│
├── security
│   └── CustomUserDetailsService.java
```

```
│
└── service
    ├── CourseService.java
    ├── CourseServiceImpl.java
    ├── StudentService.java
    └── StudentServiceImpl.java
```

---

# 9. Database Requirements

## 9.1 Database Name

The database must be named:

```
many_to_many_demo
```

## 9.2 Required Business Tables

The application must generate and use the following business tables:

| Table Name | Purpose |
|---|---|
| students | Stores student details |
| courses | Stores course details |
| student_courses | Stores student-course relationship |

## 9.3 Required Security Tables

The application must generate and use the following security tables:

| Table Name | Purpose |
|---|---|
| users | Stores application users |
| roles | Stores available roles |
| user_roles | Stores user-role relationship |

---

# 10. Business Entity Requirements

---

## 10.1 Student Entity

Class name:

```
Student
```

Package:

```
com.swabhav.demo.model
```

Purpose:

Represents a student in the system.

Required fields:

| Field Name | Data Type | Requirement |
|---|---|---|
| id | Long | Unique student identifier |
| fullName | String | Required |
| email | String | Required and unique |
| age | Integer | Required and greater than zero |
| courses | Set<Course> | Many-to-many relationship with Course |

Relationship requirement:

1. `Student` must be the owning side of the student-course relationship.
2. The join table must represent the relationship between students and courses.
3. A student may have zero, one, or many courses.
4. Duplicate course assignment must not be allowed for the same student.

Required relationship helper method names:

| Method Name | Purpose |
|---|---|
| addCourse | Assign a course to a student |
| removeCourse | Remove a course from a student |

---

## 10.2 Course Entity

Class name:

```
Course
```

Package:

```
com.swabhav.demo.model
```

Purpose:

Represents a course in the system.

Required fields:

| Field Name | Data Type | Requirement |
|---|---|---|
| id | Long | Unique course identifier |
| courseName | String | Required |
| courseCode | String | Required and unique |
| fees | Double | Required and greater than zero |
| students | Set<Student> | Many-to-many relationship with Student |

Relationship requirement:

1. `Course` must be the inverse side of the student-course relationship.
2. Deleting a course must remove student-course relationships.
3. Deleting a course must not delete students.

---

# 11. Security Entity Requirements

---

## 11.1 AppUser Entity

Class name:

```
AppUser
```

Package:

```
com.swabhav.demo.model
```

Purpose:

Represents an application user stored in the database.

Required fields:

| Field Name | Data Type | Requirement |
|---|---|---|
| id | Long | Unique user identifier |
| username | String | Required and unique |
| password | String | Required and encrypted |
| enabled | boolean | Indicates whether the user is active |
| roles | Set<Role> | Many-to-many relationship with Role |

Business rule:

1. Users must be loaded from the database during authentication.
2. Passwords must not be stored in plain text.
3. Users must receive permissions from their assigned roles.

---

## 11.2 Role Entity

Class name:

```
Role
```

Package:

```
com.swabhav.demo.model
```

Purpose:

Represents an application role.

Required fields:

| Field Name | Data Type | Requirement |
|---|---|---|
| id | Long | Unique role identifier |
| name | String | Required and unique |
| users | Set<AppUser> | Many-to-many relationship with AppUser |

Required role values:

| Role Name |
|---|
| ROLE_ADMIN |
| ROLE_USER |

---

# 12. DTO Requirements

Entities must not be directly exposed through controllers.

The system must use DTOs for request and response handling.

---

## 12.1 CourseRequestDto

Purpose:

Used to create or update course data.

Required fields:

| Field Name | Data Type | Validation |
|---|---|---|
| courseName | String | Required |
| courseCode | String | Required |
| fees | Double | Required and greater than zero |

JSON field names:

| Java Field | JSON Field |
|---|---|
| courseName | course_name |
| courseCode | course_code |

---

## 12.2 CourseResponseDto

Purpose:

Used to send course details in API responses.

Required fields:

| Field Name | Data Type |
|---|---|
| id | Long |
| courseName | String |
| courseCode | String |
| fees | Double |

JSON field names:

| Java Field | JSON Field |
|---|---|
| courseName | course_name |
| courseCode | course_code |

---

## 12.3 StudentRequestDto

Purpose:

Used to create or update student data.

Required fields:

| Field Name | Data Type | Validation |
|---|---|---|
| fullName | String | Required |
| email | String | Required and valid email |

| Field Name | Data Type | Validation |
|---|---|---|
| age | Integer | Required and greater than zero |
| courseIds | List<Long> | Optional |

JSON field names:

| Java Field | JSON Field |
|---|---|
| fullName | full_name |
| courseIds | course_ids |

Business rule:

1. `course_ids` may be empty or missing.
2. If provided, `course_ids` must not contain duplicate values.
3. If any provided course ID does not exist, the system must return a not found error.

---

## 12.4 StudentResponseDto

Purpose:

Used to send student details with assigned courses.

Required fields:

| Field Name | Data Type |
|---|---|
| id | Long |
| fullName | String |
| email | String |
| age | Integer |
| courses | List<CourseResponseDto> |

JSON field names:

| Java Field | JSON Field |
|---|---|
| fullName | full_name |

---

## 12.5 PageResponseDto

Purpose:

Used to send paginated API responses.

Generic type:

```
PageResponseDto<T>
```

Required fields:

| Field Name | Data Type |
|---|---|
| content | List<T> |
| pageNumber | int |
| pageSize | int |
| totalElements | long |
| totalPages | int |
| lastPage | boolean |

---

# 13. Repository Requirements

---

## 13.1 StudentRepository

Package:

```
com.swabhav.demo.repository
```

Purpose:

Handles database operations for students.

Required capabilities:

1. Basic CRUD operations.
2. Pagination.

3. Email duplication check.
4. Email duplication check during update.
5. Fetch students by course ID.

Required method names:

| Method Name | Purpose |
|---|---|
| existsByEmail | Check whether student email already exists |
| existsByEmailAndIdNot | Check duplicate email while updating |
| findByCoursesId | Fetch students enrolled in a course |

---

## 13.2 CourseRepository

Package:

```
com.swabhav.demo.repository
```

Purpose:

Handles database operations for courses.

Required capabilities:

1. Basic CRUD operations.
2. Pagination.
3. Course code duplication check.
4. Course code duplication check during update.
5. Fetch courses by student ID.

Required method names:

| Method Name | Purpose |
|---|---|
| existsByCourseCode | Check whether course code already exists |
| existsByCourseCodeAndIdNot | Check duplicate course code while updating |
| findByStudentsId | Fetch courses assigned to a student |

---

## 13.3 UserRepository

Package:

```
com.swabhav.demo.repository
```

Purpose:

Handles database operations for security users.

Required method names:

| Method Name | Purpose |
|---|---|
| findByUsername | Fetch user by username during authentication |
| existsByUsername | Check whether username already exists |

---

## 13.4 RoleRepository

Package:

```
com.swabhav.demo.repository
```

Purpose:

Handles database operations for roles.

Required method names:

| Method Name | Purpose |
|---|---|
| findByName | Fetch role by role name |

---

# 14. Service Layer Requirements

---

## 14.1 CourseService

Package:

```
com.swabhav.demo.service
```

Type:

```
Interface
```

Required method names:

| Method Name | Purpose |
|---|---|
| createCourse | Create a new course |
| getAllCourses | Fetch all courses |
| getAllCoursesWithPagination | Fetch courses with pagination and sorting |
| getCourseById | Fetch course by ID |
| getStudentsByCourseId | Fetch students enrolled in a course |
| updateCourse | Update course details |
| deleteCourse | Delete course |

---

## 14.2 CourseServiceImpl

Package:

```
com.swabhav.demo.service
```

Type:

```
Service implementation class
```

Required dependencies:

| Dependency | Purpose |
|---|---|
| CourseRepository | Course database operations |
| StudentRepository | Student lookup for course-student relationship APIs |
| ModelMapper | DTO mapping |

Required public methods:

| Method Name |
|---|
| createCourse |
| getAllCourses |
| getAllCoursesWithPagination |
| getCourseById |
| getStudentsByCourseId |
| updateCourse |
| deleteCourse |

Required private helper methods:

| Method Name | Purpose |
|---|---|
| findCourseById | Fetch course or throw not found exception |
| mapStudentToBasicResponseDto | Convert student to response DTO for course-student response |
| validatePagination | Validate pagination inputs |
| validateCourseSortField | Validate allowed sorting fields for course |
| getSortDirection | Validate and resolve sorting direction |

Business rules:

1. Course code must be unique.
2. Course fees must be greater than zero.
3. A course can be deleted only after removing its student-course relationships.
4. Deleting a course must not delete students.
5. Invalid sort fields must return a bad request response.

---

## 14.3 StudentService

Package:

```
com.swabhav.demo.service
```

Type:

```
Interface
```

Required method names:

| Method Name | Purpose |
|---|---|
| createStudent | Create a student with optional course assignments |
| getAllStudents | Fetch all students |
| getAllStudentsWithPagination | Fetch students with pagination and sorting |
| getStudentById | Fetch student by ID |
| getCoursesByStudentId | Fetch courses assigned to a student |
| updateStudent | Update student and assigned courses |
| assignCourseToStudent | Assign one course to one student |
| removeCourseFromStudent | Remove one course from one student |
| deleteStudent | Delete student |

---

## 14.4 StudentServiceImpl

Package:

```
com.swabhav.demo.service
```

Type:

```
Service implementation class
```

Required dependencies:

| Dependency | Purpose |
|---|---|
| StudentRepository | Student database operations |
| CourseRepository | Course database operations |
| ModelMapper | DTO mapping |

Required public methods:

| Method Name |
|---|
| createStudent |
| getAllStudents |
| getAllStudentsWithPagination |
| getStudentById |
| getCoursesByStudentId |
| updateStudent |
| assignCourseToStudent |
| removeCourseFromStudent |
| deleteStudent |

Required private helper methods:

| Method Name | Purpose |
|---|---|
| findStudentById | Fetch student or throw not found exception |
| findCourseById | Fetch course or throw not found exception |
| getCoursesFromIds | Fetch course records from course IDs |
| replaceStudentCourses | Replace assigned courses during update |
| validateDuplicateCourseIds | Prevent duplicate course IDs in request |
| mapStudentToResponseDto | Convert student to response DTO |
| validatePagination | Validate pagination inputs |
| validateStudentSortField | Validate allowed sorting fields for student |
| getSortDirection | Validate and resolve sorting direction |

Business rules:

1. Student email must be unique.
2. A student can be created without courses.
3. If course IDs are provided, all course IDs must exist.
4. Duplicate course IDs in request must not be allowed.
5. A course already assigned to a student must not be assigned again.
6. A course not assigned to a student must not be removed.
7. Deleting a student must remove student-course relationships first.
8. Deleting a student must not delete courses.
9. Invalid sort fields must return a bad request response.

---

# 15. Controller Layer Requirements

---

## 15.1 CourseController

Package:

```
com.swabhav.demo.controller
```

Base URL:

```
/api/courses
```

Required dependency:

```
CourseService
```

Required API methods:

| Method Name | HTTP Method | Endpoint | Access |
|---|---|---|---|
| createCourse | POST | /api/courses | ADMIN |
| getAllCourses | GET | /api/courses | USER, ADMIN |
| getAllCoursesWithPagination | GET | /api/courses/page | USER, ADMIN |
| getStudentsByCourseId | GET | /api/courses/{courseId}/students | USER, ADMIN |
| getCourseById | GET | /api/courses/{id} | USER, ADMIN |

| Method Name | HTTP Method | Endpoint | Access |
|---|---|---|---|
| updateCourse | PUT | /api/courses/{id} | ADMIN |
| deleteCourse | DELETE | /api/courses/{id} | ADMIN |

Important routing requirement:

The route for `/page` must be treated separately from `{id}`.

The route for `/{courseId}/students` must be treated separately from `/{id}`.

---

## 15.2 StudentController

Package:

```
com.swabhav.demo.controller
```

Base URL:

```
/api/students
```

Required dependency:

```
StudentService
```

Required API methods:

| Method Name | HTTP Method | Endpoint | Access |
|---|---|---|---|
| createStudent | POST | /api/students | ADMIN |
| getAllStudents | GET | /api/students | USER, ADMIN |
| getAllStudentsWithPagination | GET | /api/students/page | USER, ADMIN |
| getCoursesByStudentId | GET | /api/students/{studentId}/courses | USER, ADMIN |
| getStudentById | GET | /api/students/{id} | USER, ADMIN |

| Method Name | HTTP Method | Endpoint | Access |
|---|---|---|---|
| updateStudent | PUT | /api/students/{id} | ADMIN |
| assignCourseToStudent | PUT | /api/students/{studentId}/courses/{courseId} | ADMIN |
| removeCourseFromStudent | DELETE | /api/students/{studentId}/courses/{courseId} | ADMIN |
| deleteStudent | DELETE | /api/students/{id} | ADMIN |

Important routing requirement:

The route for `/page` must be treated separately from `{id}`.

The route for `/{studentId}/courses` must be treated separately from `/{id}`.

The route for `/{studentId}/courses/{courseId}` must be treated separately from delete-by-id.

---

# 16. API Requirements

---

## 16.1 Course APIs

### Create Course

| Requirement | Details |
|---|---|
| Method | POST |
| Endpoint | /api/courses |
| Access | ADMIN |
| Request DTO | CourseRequestDto |
| Response DTO | CourseResponseDto |
| Success Status | 201 Created |

### Get All Courses

| Requirement | Details |
|---|---|
| Method | GET |

| Requirement | Details |
|---|---|
| Endpoint | /api/courses |
| Access | USER, ADMIN |
| Response | List<CourseResponseDto> |

### Get Courses With Pagination and Sorting

| Requirement | Details |
|---|---|
| Method | GET |
| Endpoint | /api/courses/page |
| Access | USER, ADMIN |
| Response | PageResponseDto<CourseResponseDto> |

Request parameters:

| Parameter | Type | Default |
|---|---|---|
| pageNumber | int | 0 |
| pageSize | int | 5 |
| sortBy | String | id |
| sortDirection | String | asc |

Allowed sort fields:

| Field |
|---|
| id |
| courseName |
| courseCode |
| fees |

Allowed sort directions:

| Direction |
|---|
| asc |
| desc |

### Get Course By ID

| Requirement | Details |
|---|---|
| Method | GET |
| Endpoint | /api/courses/{id} |
| Access | USER, ADMIN |
| Response | CourseResponseDto |

### Get Students By Course ID

| Requirement | Details |
|---|---|
| Method | GET |
| Endpoint | /api/courses/{courseId}/students |
| Access | USER, ADMIN |
| Response | List<StudentResponseDto> |

### Update Course

| Requirement | Details |
|---|---|
| Method | PUT |
| Endpoint | /api/courses/{id} |
| Access | ADMIN |
| Request DTO | CourseRequestDto |
| Response DTO | CourseResponseDto |

### Delete Course

| Requirement | Details |
|---|---|
| Method | DELETE |
| Endpoint | /api/courses/{id} |
| Access | ADMIN |
| Success Status | 204 No Content |

---

## 16.2 Student APIs

### Create Student

| Requirement | Details |
|---|---|
| Method | POST |
| Endpoint | /api/students |
| Access | ADMIN |
| Request DTO | StudentRequestDto |
| Response DTO | StudentResponseDto |
| Success Status | 201 Created |

### Get All Students

| Requirement | Details |
|---|---|
| Method | GET |
| Endpoint | /api/students |
| Access | USER, ADMIN |
| Response | List<StudentResponseDto> |

### Get Students With Pagination and Sorting

| Requirement | Details |
|---|---|
| Method | GET |
| Endpoint | /api/students/page |
| Access | USER, ADMIN |
| Response | PageResponseDto<StudentResponseDto> |

Request parameters:

| Parameter | Type | Default |
|---|---|---|
| pageNumber | int | 0 |
| pageSize | int | 5 |
| sortBy | String | id |

| Parameter | Type | Default |
|---|---|---|
| sortDirection | String | asc |

Allowed sort fields:

| Field |
|---|
| id |
| fullName |
| email |
| age |

Allowed sort directions:

| Direction |
|---|
| asc |
| desc |

### Get Student By ID

| Requirement | Details |
|---|---|
| Method | GET |
| Endpoint | /api/students/{id} |
| Access | USER, ADMIN |
| Response | StudentResponseDto |

### Get Courses By Student ID

| Requirement | Details |
|---|---|
| Method | GET |
| Endpoint | /api/students/{studentId}/courses |
| Access | USER, ADMIN |
| Response | List<CourseResponseDto> |

### Update Student

| Requirement | Details |
|---|---|
| Method | PUT |
| Endpoint | /api/students/{id} |
| Access | ADMIN |
| Request DTO | StudentRequestDto |
| Response DTO | StudentResponseDto |

### Assign Course To Student

| Requirement | Details |
|---|---|
| Method | PUT |
| Endpoint | /api/students/{studentId}/courses/{courseId} |
| Access | ADMIN |
| Response | StudentResponseDto |

Business rule:

The same course must not be assigned again to the same student.

### Remove Course From Student

| Requirement | Details |
|---|---|
| Method | DELETE |
| Endpoint | /api/students/{studentId}/courses/{courseId} |
| Access | ADMIN |
| Response | StudentResponseDto |

Business rule:

The system must not allow removal of a course that is not assigned to the student.

### Delete Student

| Requirement | Details |
|---|---|
| Method | DELETE |

| Requirement | Details |
|---|---|
| Endpoint | /api/students/{id} |
| Access | ADMIN |
| Success Status | 204 No Content |

---

# 17. Validation Requirements

## 17.1 Course Validation

| Field | Rule |
|---|---|
| courseName | Required |
| courseCode | Required and unique |
| fees | Required and greater than zero |

## 17.2 Student Validation

| Field | Rule |
|---|---|
| fullName | Required |
| email | Required, valid email, and unique |
| age | Required and greater than zero |
| courseIds | Optional, but must not contain duplicate values if provided |

## 17.3 Pagination Validation

| Field | Rule |
|---|---|
| pageNumber | Must not be negative |
| pageSize | Must be greater than zero |
| pageSize | Must not be greater than 100 |
| sortBy | Must be one of the allowed fields |
| sortDirection | Must be either asc or desc |

---

# 18. Exception Handling Requirements

The system must return consistent error responses.

## 18.1 Required Exception Classes

| Exception Class | Purpose |
|---|---|
| ResourceNotFoundException | Used when student, course, user, or role is not found |
| DuplicateResourceException | Used for duplicate student email or course code |
| BadRequestException | Used for invalid business requests |
| GlobalExceptionHandler | Handles exceptions globally |

---

## 18.2 Required Global Exception Handler Methods

| Method Name | Handles |
|---|---|
| handleResourceNotFound | ResourceNotFoundException |
| handleDuplicateResource | DuplicateResourceException |
| handleBadRequest | BadRequestException |
| handleIllegalArgument | IllegalArgumentException |
| handleValidation | MethodArgumentNotValidException |
| handleTypeMismatch | MethodArgumentTypeMismatchException |
| handleInvalidJson | HttpMessageNotReadableException |
| handleDataIntegrity | DataIntegrityViolationException |
| handleAccessDenied | AccessDeniedException |
| handleGeneric | Exception |

---

## 18.3 Standard Error Response Format

Normal error response must include:

| Field |
|---|
| timestamp |

| Field |
| --- |
| status |
| error |
| message |

Validation error response must include:

| Field |
| --- |
| timestamp |
| status |
| error |
| messages |

---

## 18.4 Required Status Codes

| Scenario | Status |
| --- | --- |
| Resource not found | 404 Not Found |
| Duplicate resource | 409 Conflict |
| Invalid business request | 400 Bad Request |
| Validation failure | 400 Bad Request |
| Invalid JSON | 400 Bad Request |
| Invalid path variable or query parameter | 400 Bad Request |
| Access denied | 403 Forbidden |
| Unexpected error | 500 Internal Server Error |

---

# 19. Security Requirements

## 19.1 Authentication

The project must use:

```
HTTP Basic Authentication
```

The project must not use JWT.

---

## 19.2 User and Role Storage

Users and roles must be stored in the database.

Required tables:

| Table |
|---|
| users |
| roles |
| user_roles |

---

## 19.3 Default Users

The project must create default users for development/testing.

| Username | Password | Roles |
|---|---|---|
| admin | admin123 | ROLE_ADMIN, ROLE_USER |
| user | user123 | ROLE_USER |

Passwords must be stored in encrypted form.

---

## 19.4 Role-Based Access Rules

| API Pattern | HTTP Method | Access |
|---|---|---|
| /api/students/** | GET | USER, ADMIN |
| /api/students/** | POST | ADMIN |
| /api/students/** | PUT | ADMIN |
| /api/students/** | DELETE | ADMIN |
| /api/courses/** | GET | USER, ADMIN |

| API Pattern | HTTP Method | Access |
|---|---|---|
| /api/courses/** | POST | ADMIN |
| /api/courses/** | PUT | ADMIN |
| /api/courses/** | DELETE | ADMIN |

---

## 19.5 Public Endpoints

The following endpoints must be public:

| Endpoint |
|---|
| /swagger-ui.html |
| /swagger-ui/** |
| /v3/api-docs/** |

---

## 19.6 Security Configuration Requirements

Security configuration must include:

1. Database-based authentication.
2. Password encryption.
3. Stateless session policy.
4. CSRF disabled for REST API testing.
5. Swagger endpoints publicly accessible.
6. Role-based authorization.
7. HTTP Basic enabled.

---

# 20. Swagger Requirements

Swagger must be available at:

```
http://localhost:8080/swagger-ui.html
```

OpenAPI JSON must be available at:

```
http://localhost:8080/v3/api-docs
```

Swagger must show:

1. Student APIs.
2. Course APIs.
3. Basic Auth authorization option.
4. Request DTO schemas.
5. Response DTO schemas.
6. Pagination parameters.
7. Sorting parameters.

Swagger documentation title:

```
Student Course Many-to-Many API
```

Swagger version:

```
1.0
```

---

# 21. Logging Requirements

The project must include logger statements in:

| Class | Logging Requirement |
|---|---|
| StudentController | Log student API requests |
| CourseController | Log course API requests |
| StudentServiceImpl | Log student business operations |
| CourseServiceImpl | Log course business operations |
| CustomUserDetailsService | Log user loading and user-not-found cases |
| GlobalExceptionHandler | Log exceptions and errors |

Required log situations:

| Situation | Level |
|---|---|
| API request received | INFO |

| Situation | Level |
|---|---|
| Create operation started | INFO |
| Create operation completed | INFO |
| Read operation | INFO |
| Update operation started | INFO |
| Update operation completed | INFO |
| Delete operation started | INFO |
| Delete operation completed | INFO |
| Duplicate resource | WARN |
| Resource not found | WARN |
| Invalid business request | WARN |
| Validation failure | WARN |
| Access denied | WARN |
| Database constraint error | ERROR |
| Unexpected system error | ERROR |

Sensitive information such as passwords must never be logged.

---

# 22. Configuration Requirements

## 22.1 Application Configuration

The project must include configuration for:

| Property Purpose |
|---|
| Database URL |
| Database username |
| Database password |
| Hibernate DDL mode |
| SQL visibility |
| SQL formatting |

| Property Purpose |
| --- |
| Server port |
| Swagger UI path |
| Logging level |

Server port:

```
8080
```

---

## 22.2 AppConfig

Purpose:

Application-level bean configuration.

Required bean:

| Bean |
| --- |
| ModelMapper |

---

## 22.3 SecurityConfig

Purpose:

Security configuration.

Must configure:

1. Security filter chain.
2. Authentication provider.
3. Password encoder.
4. HTTP Basic authentication.
5. Role-based authorization.
6. Database-based user authentication.

---

## 22.4 OpenApiConfig

Purpose:

Swagger/OpenAPI configuration.

Must configure:

1. API title.
2. API version.
3. API description.
4. Basic Auth security scheme.

---

## 22.5 DataInitializer

Purpose:

Create default roles and users in the database for development/testing.

Must create:

| Data |
|---|
| ROLE_ADMIN |
| ROLE_USER |
| admin user |
| user user |

The initializer must not create duplicate users or roles if they already exist.

---

# 23. Sample Request Requirements

## 23.1 Create Course Request

Required JSON fields:

| Field |
|---|
| course_name |
| course_code |

| Field |
| --- |
| fees |

## 23.2 Create Student Request

Required JSON fields:

| Field |
| --- |
| full_name |
| email |
| age |
| course_ids |

`course_ids` may be empty or omitted.

## 23.3 Update Student Request

Required JSON fields:

| Field |
| --- |
| full_name |
| email |
| age |
| course_ids |

The update operation must replace the student's course list with the provided course IDs.

---

# 24. Testing Requirements

The project must be tested using:

1. Swagger UI.
2. Postman.
3. MySQL database verification.

---

## 24.1 Required Positive Test Cases

| Test Case | Expected Result |
|---|---|
| Create course as ADMIN | Success |
| Create student as ADMIN | Success |
| Assign course to student as ADMIN | Success |
| Remove course from student as ADMIN | Success |
| Get all students as USER | Success |
| Get all courses as USER | Success |
| Get courses by student ID | Success |
| Get students by course ID | Success |
| Student pagination with sorting | Success |
| Course pagination with sorting | Success |
| Delete student as ADMIN | Success |
| Delete course as ADMIN | Success |

---

## 24.2 Required Negative Test Cases

| Test Case | Expected Result |
|---|---|
| Create duplicate student email | Conflict error |
| Create duplicate course code | Conflict error |
| Create student with duplicate course IDs | Bad request error |
| Assign same course twice to student | Bad request error |
| Remove unassigned course from student | Bad request error |
| Fetch non-existing student | Not found error |
| Fetch non-existing course | Not found error |
| USER tries to create course | Forbidden |
| USER tries to delete student | Forbidden |
| Invalid page number | Bad request |
| Invalid page size | Bad request |

| Test Case | Expected Result |
|---|---|
| Invalid sort field | Bad request |
| Invalid sort direction | Bad request |
| Invalid JSON body | Bad request |
| Missing required fields | Validation error |

---

# 25. Final Expected Deliverables

Mentees must submit:

1. Complete Spring Boot project.
2. Clean package structure.
3. MySQL database configured.
4. Student-course many-to-many mapping working.
5. User-role database security working.
6. CRUD APIs for students.
7. CRUD APIs for courses.
8. Assign-course API working.
9. Remove-course API working.
10. Pagination and sorting working.
11. Swagger UI working.
12. Global exception handling working.
13. Logging added.
14. Validation working.
15. Role-based authorization working.
16. Postman collection or API testing screenshots.
17. Database screenshots showing generated tables.

---

# 26. Completion Criteria

The project will be considered complete only if:

1. Students can be created.
2. Courses can be created.
3. Students can be assigned to multiple courses.
4. Courses can have multiple students.
5. Student-course relationships are stored in the join table.
6. Student deletion removes only student-course relationships and student record.
7. Course deletion removes only student-course relationships and course record.
8. Courses are not deleted when students are deleted.

9. Students are not deleted when courses are deleted.
10. Users and roles are stored in the database.
11. Login works using database users.
12. Role-based API access works correctly.
13. Pagination works for students and courses.
14. Sorting works for students and courses.
15. Swagger works.
16. Exception responses are clean and consistent.
17. Logs are visible for important operations.
18. DTOs are used instead of exposing entities directly.
19. No JWT is used.
20. Code follows the required layered architecture.

---

# 27. Recommended Implementation Order

Mentees should implement the project in this order:

1. Create Spring Boot project.
2. Add required dependencies.
3. Configure application properties.
4. Create MySQL database.
5. Create business entities.
6. Create security entities.
7. Create DTOs.
8. Create repositories.
9. Create custom exceptions.
10. Create global exception handler.
11. Create configuration classes.
12. Create database user-role initializer.
13. Create custom user details service.
14. Create security configuration.
15. Create service interfaces.
16. Create service implementations.
17. Create controllers.
18. Add Swagger configuration.
19. Add logging.
20. Test course APIs.
21. Test student APIs.
22. Test assignment and removal APIs.
23. Test pagination and sorting.
24. Test role-based security.
25. Test exception cases.
26. Verify database tables and relationships.